

Laboratorio di Programmazione

(Laurea triennale in matematica)

Lezione 2

Analisi dettagliata di un programma

`/* commento */`

Possono estendersi su più linee apparire in qualsiasi parte del programma.

Alternativamente

`// commento`

in questo caso possono estendersi su una sola linea.

```
/*Calcolo area rettangolo*/  
#include <stdio.h>
```

```
int main(void)  
{  
    int base;  
    int altezza;  
    int area;  
  
    base = 3;  
    altezza = 7;  
    area = base*altezza;  
  
    printf("%d\n", area);  
}
```

Variabili e assegnamenti

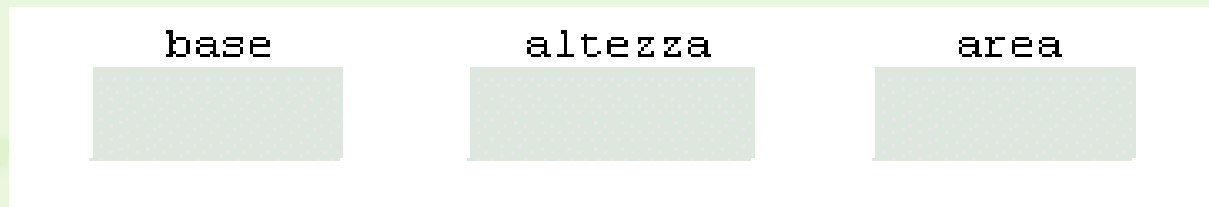
Dichiarazioni di variabili intere

```
int base;  
int altezza;  
int area;
```

`int` specifica che si tratta di una variabile numerica di tipo **intero**.

Come le altre istruzioni devono terminare con un punto e virgola.

Nell'esempio, a ogni dichiarazione di variabile corrisponde anche la sua **definizione**: le viene riservato uno spazio adeguato in memoria centrale:



Variabili e assegnamenti

Il **nome** di una variabile la identifica

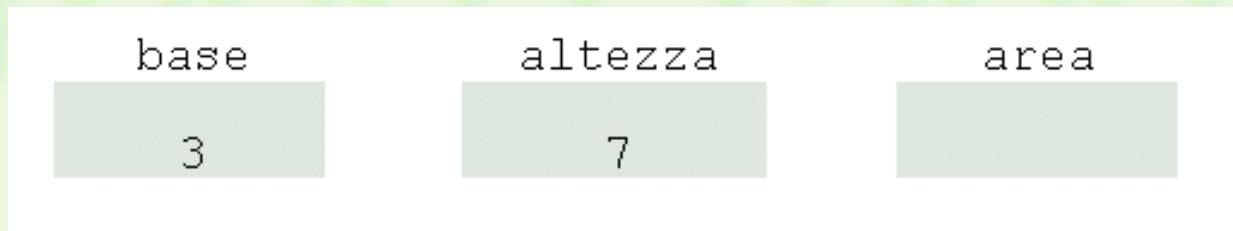
il suo **tipo** ne definisce la dimensione e l'insieme delle operazioni che si possono effettuare su di essa.

La dimensione può variare rispetto all'implementazione

```
base = 3;
```

assegna alla variabile **base** il valore 3.

L'assegnamento è dunque realizzato mediante l'operatore **=**.



Variabili e assegnamenti

```
area = base*altezza;
```

assegna alla variabile `area` il prodotto dei valori di `base` e `altezza`.

base	altezza	area
3	7	21

mentre l'ultima istruzione

```
printf("%d\n", area);
```

visualizza 21, il valore della variabile `area`.

% specifica che il carattere seguente (`d`) definisce il formato di stampa di `area`.

`d` (***decimal***) indica il formato di stampa: un intero nel sistema decimale.

`\n` provoca un salto a linea nuova dopo la visualizzazione.

Input e output

Per immettere valori:

```
scanf ("%d", &base);
```

%d indica un valore intero in formato decimale assegnato alla variabile **base**. **&base** indica l'indirizzo di memoria in cui si trova la variabile **base**.

```
printf("Valore della base: ");
```

```
scanf ("%d", &base);
```

Quello che apparirà all'*utente* sarà:

Valore della base:

se l'utente digita 15:

Valore della base: 15 Invio



questo verrà assegnato alla variabile **base**

Input e Output

Nell'ipotesi che l'utente inserisca i valori 10 e 13 avremo:

AREA RETTANGOLO

Valore base: 10

Valore altezza: 13

Base: 10

Altezza: 13

Area: 130

```
/* Calcolo area rettangolo */
#include <stdio.h>

int base, altezza, area;
int main(void)
{
    printf("AREA RETTANGOLO\n\n");
    printf("Valore base: ");
    scanf("%d", &base);
    printf("Valore altezza: ");
    scanf("%d", &altezza);

    area = base*altezza;

    printf("Base: %d\n", base);
    printf("Altezza: %d\n", altezza);
    printf("Area: %d\n", area);
}
```

Input e output

Per visualizzare più variabili con una sola `printf`

```
printf("%d %d %d", base, altezza, area);
```

degli a-capo a piacere:

```
printf("%d\n%d\n%d", base, altezza, area);
```

dei commenti:

```
printf("Base: %d\nAltezza: %d\nArea: %d", base, altezza, area);
```

Sequenze di escape, sequenze di caratteri con funzioni speciali:

<code>\n</code>	va a linea nuova
<code>\t</code>	salta di una tabulazione
<code>\b</code>	ritorna un carattere indietro (<i>backspace</i>)
<code>\a</code>	suona il campanello della macchina
<code>\\</code>	stampa il carattere <code>\</code>
<code>\"</code>	stampa il carattere <code>"</code>

```
/* ESEMPIO: scambio di variabili*/
#include <stdio.h>
int main()
{
    int num1, num2, a;

    printf ("Scrivi il primo numero  ");
    scanf ("%d", &num1);
    printf ("Scrivi il secondo  numero  ");
    scanf ("%d", &num2);

    printf("\n num1= %d \t num2=%d", num1,num2);
    printf("\n Adesso effettuo lo scambio! \n");

    a= num1;
    num1=num2;
    num2=a;

    printf("\n\n\n num1= %d \t num2=%d", num1,num2);

    return (0);
}
```

Struttura generale di un programma

```
inclusioneDiLibrerie
```

```
int main(void)
```

```
{
```

```
    dichiarazioneDiVariabili
```

```
    istruzione1
```

```
    istruzione2
```

```
    istruzione3
```

```
    ...
```

```
    istruzioneN
```

```
}
```

NOTA: Nella sintassi il punto e virgola fa parte dell'istruzione stessa.

Identificatori

- Nomi per le variabili (e le funzioni)
- Scelti dal programmatore
- Possono contenere numeri e lettere maiuscole e minuscole (sono diverse!) e underscore _
- Devono iniziare per lettera
(di solito minuscola)
- Non devono essere parole riservate o nomi di funzione

Dichiarazioni di variabili

- Ogni variabile DEVE essere dichiarata prima del suo utilizzo.
- Sintassi:
 <tipo> <identificatore>;
 <tipo> <identif_1>, ... <identif_k>;
- Possono contenere numeri e lettere maiuscole e minuscole (sono diverse!) e underscore _
- Devono iniziare per lettera

Esempio

```
int num, somma;  
int valore1;  
char risposta;  
float val_1, val_2;
```

Tipi di dati fondamentali in C

- 5 tipi di dati elementari:
 - `char` (1 byte)
 - `int` (2 o 4 byte)
 - `float` (4 byte)
 - `double` (8 byte)
 - `(void)`
- Occupazione in memoria dipende dalla macchina e dal compilatore istallato.

Cosa sono i byte???

(breve digressione sulla rappresentazione
dei dati)

Codifica dei dati (e dei programmi)

20

- Ogni informazione ha una rappresentazione digitale che utilizza due stati che dal punto di vista logico vengono rappresentati con 1 e 0 e dal punto di vista fisico corrispondono alla presenza di una tensione alta e tensione bassa.
- L'unità digitale viene chiamata *bit*.
- Il *byte* pari a **parola di 8 bit** può rappresentare 2^8 differenti sequenze di informazioni

Rappresentazione dei numeri

NATURALI:

- Base decimale (cifre da 0 a 9);
- Sistema binario (cifre 0 e 1);
- Sistema ottale ed esadecimale.

INTERI:

- Base decimale (il numero preceduto dal segno);
- Sistema binario il numero preceduto da 0 se è positivo e 1 se è negativo.

Se si hanno a disposizione m bit per rappresentare un intero allora si possono rappresentare I numeri interi compresi fra $-(2^{m-1} - 1)$ e $+(2^{m-1} - 1)$

Rappresentazione dei numeri

$$365_{10} = 3 \times 10^2 + 6 \times 10^1 + 5 \times 10^0$$

$$365_{10} = 256 + 64 + 32 + 8 + 4 + 1 =$$

$$\begin{aligned} &= 1 \times 2^8 + 0 \times 2^7 + 1 \times 2^6 + 1 \times 2^5 + 0 \times 2^4 + \\ &\quad 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = \\ &= 101101101_2 \end{aligned}$$

Rappresentazione dei numeri

- Rappresentazione esadecimale

(alfabeto di 16 caratteri: 0, 1, ..., 9, A, B, C, D, E, F)

$$15_{10} = F_{16}$$

$$16_{10} = 1 \times 16^1 + 0 \times 16^0 = 10_{16}$$

$$365_{10} = 16D_{16}$$

- "Vantaggi" dell'esadecimale:

(Una cifra esadecimale $\longrightarrow$ numero binario di 4 cifre)

$$A16B_{16} = 1010\ 0001\ 0110\ 1011_2$$

$$100101_2 = 0010\ 0101_2 = 25_{16} = 37_{10}$$

Tipi di dati fondamentali in C

- 5 tipi di dati elementari:
 - `char` (1 byte)
 - `int` (2 o 4 byte)
 - `float` (4 byte)
 - `double` (8 byte)
 - `(void)`
- Occupazione in memoria dipende dalla macchina e dal compilatore istallato.

Rappresentazione dei caratteri

I caratteri di un testo vengono tradotti mediante sequenze di bit tramite un codice di traduzione.

Ciascun carattere occupa nel calcolatore 1 byte

Il più usato è il **codice ASCII** (American Standard Code for Information Interchange) a 7 bit quindi un massimo di 128 caratteri.

- Caratteri di comando: la loro scrittura su una periferica non determina una stampa ma una esecuzione di un comando. Per esempio “andare a capo”.
- Caratteri alfanumerici: maiuscole da A a Z, minuscole da a a z, cifre da 0 a 9.
I simboli : operatori aritmetici e punteggiatura (Mancano dei caratteri, per esempio le lettere accentate e le lettere greche).

Varianti dell'ASCII: usano l'ottavo bit per rappresentare altri 128 caratteri. (UTF-8, UNICODE)

Caratteri stampabili

Binary	Oct	Dec	Hex	Glyph	Binary	Oct	Dec	Hex	Glyph	Binary	Oct	Dec	Hex	Glyph
010 0000	040	32	20		100 0000	100	64	40	@	110 0000	140	96	60	`
010 0001	041	33	21	!	100 0001	101	65	41	A	110 0001	141	97	61	a
010 0010	042	34	22	"	100 0010	102	66	42	B	110 0010	142	98	62	b
010 0011	043	35	23	#	100 0011	103	67	43	C	110 0011	143	99	63	c
010 0100	044	36	24	\$	100 0100	104	68	44	D	110 0100	144	100	64	d
010 0101	045	37	25	%	100 0101	105	69	45	E	110 0101	145	101	65	e
010 0110	046	38	26	&	100 0110	106	70	46	F	110 0110	146	102	66	f
010 0111	047	39	27	'	100 0111	107	71	47	G	110 0111	147	103	67	g
010 1000	050	40	28	(	100 1000	110	72	48	H	110 1000	150	104	68	h
010 1001	051	41	29	)	100 1001	111	73	49	I	110 1001	151	105	69	i
010 1010	052	42	2A	*	100 1010	112	74	4A	J	110 1010	152	106	6A	j
010 1011	053	43	2B	+	100 1011	113	75	4B	K	110 1011	153	107	6B	k
010 1100	054	44	2C	,	100 1100	114	76	4C	L	110 1100	154	108	6C	l
010 1101	055	45	2D	-	100 1101	115	77	4D	M	110 1101	155	109	6D	m
010 1110	056	46	2E	.	100 1110	116	78	4E	N	110 1110	156	110	6E	n
010 1111	057	47	2F	/	100 1111	117	79	4F	O	110 1111	157	111	6F	o
011 0000	060	48	30	0	101 0000	120	80	50	P	111 0000	160	112	70	p
011 0001	061	49	31	1	101 0001	121	81	51	Q	111 0001	161	113	71	q
011 0010	062	50	32	2	101 0010	122	82	52	R	111 0010	162	114	72	r
011 0011	063	51	33	3	101 0011	123	83	53	S	111 0011	163	115	73	s
011 0100	064	52	34	4	101 0100	124	84	54	T	111 0100	164	116	74	t
011 0101	065	53	35	5	101 0101	125	85	55	U	111 0101	165	117	75	u
011 0110	066	54	36	6	101 0110	126	86	56	V	111 0110	166	118	76	v
011 0111	067	55	37	7	101 0111	127	87	57	W	111 0111	167	119	77	w
011 1000	070	56	38	8	101 1000	130	88	58	X	111 1000	170	120	78	x
011 1001	071	57	39	9	101 1001	131	89	59	Y	111 1001	171	121	79	y
011 1010	072	58	3A	:	101 1010	132	90	5A	Z	111 1010	172	122	7A	z
011 1011	073	59	3B	;	101 1011	133	91	5B	[	111 1011	173	123	7B	{
011 1100	074	60	3C	<	101 1100	134	92	5C	\	111 1100	174	124	7C	
011 1101	075	61	3D	=	101 1101	135	93	5D	]	111 1101	175	125	7D	}
011 1110	076	62	3E	>	101 1110	136	94	5E	^	111 1110	176	126	7E	~
011 1111	077	63	3F	?	101 1111	137	95	5F	_					

Caratteri di controllo

Binary	Dec	Hex	Abbr	C	Description
000 0000	0	00	NUL	\0	Null character
000 0001	1	01	SOH		Start of Header
000 0010	2	02	STX		Start of Text
000 0011	3	03	ETX		End of Text
000 0100	4	04	EOT		End of Transmission
000 0101	5	05	ENQ		Enquiry
000 0110	6	06	ACK		Acknowledgment
000 0111	7	07	BEL	\a	Bell
000 1000	8	08	BS	\b	Backspace
000 1001	9	09	HT	\t	Horizontal Tab
000 1010	10	0A	LF	\n	Line feed
000 1011	11	0B	VT	\v	Vertical Tab
000 1100	12	0C	FF	\f	Form feed
000 1101	13	0D	CR	\r	Carriage return
000 1110	14	0E	SO		Shift Out
000 1111	15	0F	SI		Shift In
001 0000	16	10	DLE		Data Link Escape
001 0001	17	11	DC1		Device Control 1 (oft. XON)
001 0010	18	12	DC2		Device Control 2
001 0011	19	13	DC3		Device Control 3 (oft. XOFF)
001 0100	20	14	DC4		Device Control 4
001 0101	21	15	NAK		Negative Acknowledgment
001 0110	22	16	SYN		Synchronous Idle
001 0111	23	17	ETB		End of Trans. Block
001 1000	24	18	CAN		Cancel
001 1001	25	19	EM		End of Medium
001 1010	26	1A	SUB		Substitute
001 1011	27	1B	ESC	\e	Escape
001 1100	28	1C	FS		File Separator
001 1101	29	1D	GS		Group Separator
001 1110	30	1E	RS		Record Separator
001 1111	31	1F	US		Unit Separator
111 1111	127	7F	DEL		Delete

Caratteri

Le variabili di tipo carattere assumono valori alfanumerici: lettere dell'alfabeto, cifre decimali, punteggiatura etc. Per dichiararle

```
char x, y, z;
```

La definizione riserva uno spazio sufficiente per un carattere. Per assegnargli un valore:

```
x = 'A';  
y = ';';  
z = '&';
```

Per assegnare un valore costante a una variabile `char` lo si racchiude tra apici singoli

Caratteri

Per visualizzare dei **char**:

```
printf("%c %c %c", x, y, z);
```

Per immettere caratteri:

```
scanf ("%c", &x)
```

Interi

```
int indice;
```

int specifica che si tratta di una variabile numerica di tipo *intero*.

La dimensione di un **int** può variare rispetto all'implementazione.

Alcune versioni del C riservano agli int uno spazio di **4 byte**, il che permette di poter lavorare su interi che vanno da:

−2147483648 a +2147483647.

Virgola mobile

Le variabili che possono contenere valori numerici con una parte frazionaria sono dette di tipo **float** un tipo in *virgola mobile* (*floating point*). Per dichiararle

```
float x, y, z;
```

La definizione fa sì che venga riservato uno spazio in memoria la cui dimensione dipende dall'implementazione, se 4 byte (32 bit) può contenere numeri nell'intervallo $3.4E-38$ a $3.4E+38$. Per assegnargli un valore:

```
x = 152.23; y = 0.00008; z = 7E+20;
```

Le lettera E o e indica che il numero che la precede deve essere moltiplicato per 10 elevato al numero che la segue.

Tipi in virgola mobile

Per visualizzare una variabile `float` nella `printf`

`%f` per averlo in virgola mobile,

`%e` per il formato esponenziale (notazione scientifica).

Un intero dopo `%` se presente specifica la lunghezza del campo

Un successivo punto seguito da un altro intero specifica il numero di caratteri del campo riservati alla parte decimale. Per esempio

```
printf("%15.5f", x);
```

Se `x` contiene `152.23` verrà utilizzato un campo di 15 caratteri di cui 5 dopo il punto decimale (un carattere – dopo il simbolo di percentuale indicherebbe che il valore deve essere allineato a sinistra):

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
						1	5	2	.	2	3	0	0	0

Virgola mobile

double ha capacità maggiore o uguale al tipo **float**.

Nel caso per esempio d'implementazioni con 8 byte (64 bit) copre l'intervallo da $1.7E-308$ a $1.7E+308$.

Per visualizzare un **double** con **printf** si utilizza **%lf** (*long float*).

```
scanf ("%lf", &x) ;
```

memorizza il valore nella variabile **double x**.

Operatore sizeof ()

```
int num;
```

```
sizeof(num) ;
```

```
sizeof(char) ;
```

Si può applicare sia a una *variabile* che a un *tipo* e restituisce il numero di *byte* allocati.

Operatore di Assegnamento

- Si utilizza il **simbolo =**
- Il valore alla destra di = viene assegnato alla variabile a sinistra di =
- Esempi:

```
num = 57;  
valore = 34,5;  
risp = 's';  
  
num = valore;
```

Operatore cast

```
int num1, num2;
```

```
float media;
```

```
media = (float) (num1 + num2) / 2;
```

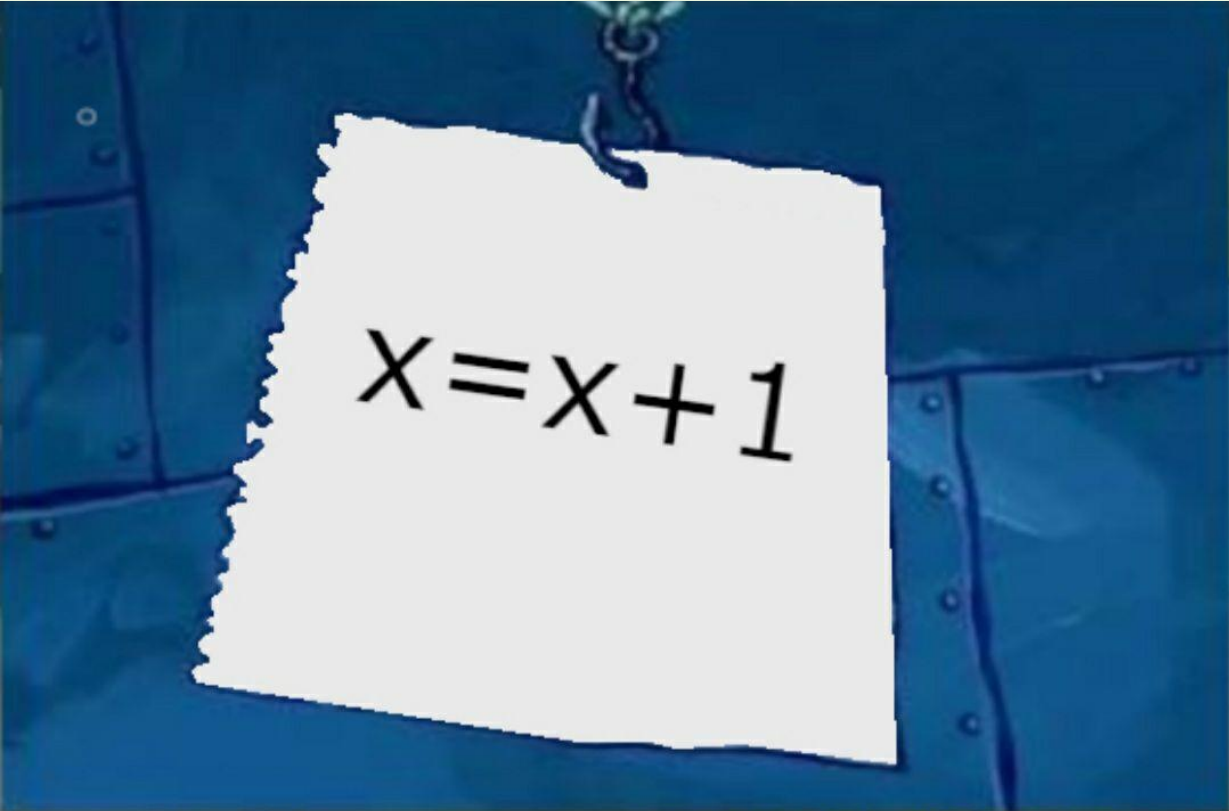
Applica una conversione di tipo.

Operatori Aritmetici

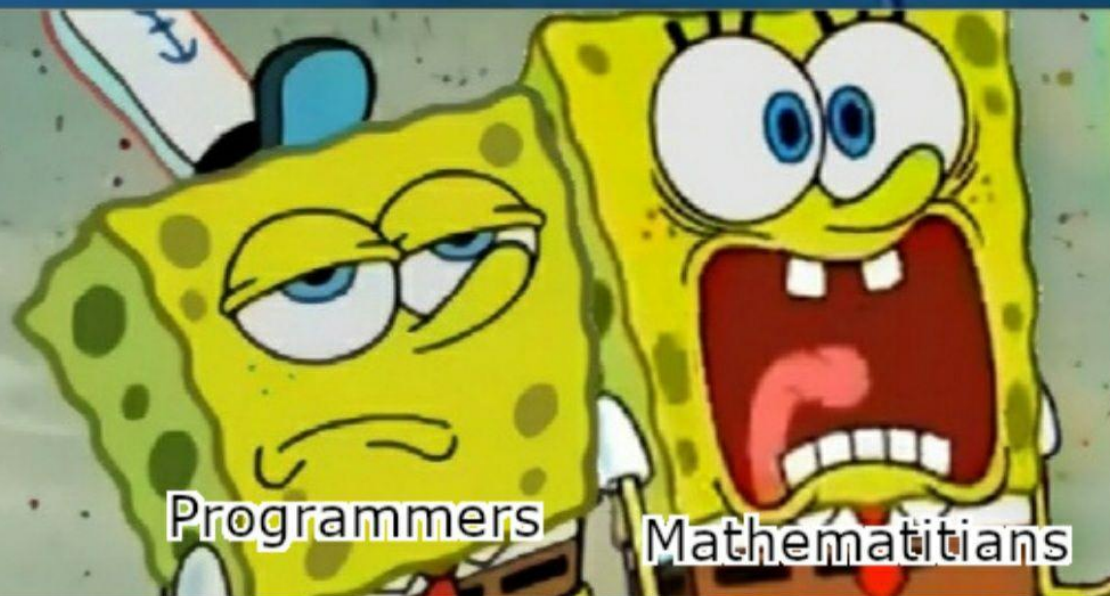
- **Espressioni aritmetiche** sono formate usando gli operatori **+**, **-**, *****, **/**, **%** insieme a variabili e numeri (*operandi*). Il risultato è un numero.
 - Se entrambi gli operandi sono dello stesso tipo, il risultato è di quel tipo.
 - Le operazioni *****, **/**, **%** hanno priorità rispetto a **+**, **-**.
 - L'ordine di priorità può essere alterato usando le parentesi **(,)**.

- Esempi:

```
num = (57 + 3) * 23 ;  
num = num +1 ;
```



$X = X + 1$



Programmers

Mathematitians